Projektdokumentation: NFA Visualisierung mit Subset Construction

Projekt: Visualizing Lexical Analysis - Subset Construction
Version: 1.0.0
Datum: Januar 2026
Autoren: Projektteam

---

1. Projektziele

1.1 Hauptziel

Entwicklung einer interaktiven Webanwendung zur Visualisierung der Umwandlung von Nichtdeterministischen Endlichen Automaten (NFA) in Deterministische Endliche Automaten (DFA) mittels Subset Construction Algorithmus.

1.2 Lernziele

- **Didaktisches Tool** für Studierende der Theoretischen Informatik
- **Visualisierung** komplexer Algorithmen der lexikalischen Analyse
- **Interaktive Exploration** von Automatentheorie-Konzepten

1.3 Funktionale Anforderungen

1. Import von NFA-Definitionen im AEF-Format (maximal 1 MB)
2. Visuelle Darstellung von NFAs als gerichtete Graphen
3. Schrittweise Ausführung des Subset Construction Algorithmus
4. Export der Ergebnisse (DFA-Definition)
5. Unterstützung von Epsilon-Transitionen (ε-Übergänge)

---

2. Systemarchitektur

2.1 Architektur-Übersicht

```
┌──────────────────────────────────────────────────────────────┐
│                      Presentation Layer                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐        │
│  │  DSLInput    │  │ GraphViewer  │  │StepControls  │        │
│  │ (Upload)     │  │(Visualizer)  │  │  (Stepper)   │        │
│  └──────────────┘  └──────────────┘  └──────────────┘        │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│                      Business Logic Layer                    │
│  ┌──────────────────┐          ┌──────────────────────┐      │
```

```
│   │   dslParser.ts    │──────────→│subsetConstruction   │          │
│   │  (AEF Parser)      │           │     (Algorithmus)   │          │
│   └────────────────────┘           └─────────────────────┘          │
└─────────────────────────────────────────────────────────────────────┘
                                  ↓
┌─────────────────────────────────────────────────────────────────────┐
│                              Data Layer                             │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐           │
│   │     NFA      │   │     DFA      │   │ Transition   │           │
│   │   (Model)    │   │   (Model)    │   │   (Model)    │           │
│   └──────────────┘   └──────────────┘   └──────────────┘           │
└─────────────────────────────────────────────────────────────────────┘
```

2.2 Technologie-Stack

Frontend

- **React 19.1.1** - UI-Framework
- **TypeScript 5.9.3** - Typsicherheit
- **Vite 7.2.2** - Build-Tool & Dev-Server
- **TailwindCSS 4.1.0** - Styling
- **ReactFlow 11.11.4** - Graph-Visualisierung
- **Dagre 0.8.5** - Automatisches Graph-Layout

Testing

- **Vitest 2.1.8** - Test-Framework
- **@testing-library/react** - Component Testing
- **@testing-library/jest-dom** - DOM-Assertions

DevOps

- **ESLint** - Code-Qualität
- **GitLab CI/CD** - Continuous Integration
- **Node 18+** - Runtime-Umgebung

---

3. Komponenten-Übersicht

3.1 Kernkomponenten

DSLInput.tsx

Zweck: Import und Validierung von NFA-Definitionen

Features:

- Datei-Upload (.aef, .dsl, .txt) bis 1 MB
- Beispiel-Vorlagen laden
- Live-Textbearbeitung

- Format-Validierung
- Fehlerberichterstattung mit Zeilennummern

Test-Coverage: 86.79% Lines, 66.66% Branches

GraphViewer.tsx

Zweck: Visuelle Darstellung von NFAs/DFAs

Features:

- Automatisches Layout mit Dagre
- Zustandsknoten mit visuellen Markierungen (Start/Accept)
- Kantenbeschriftung mit Symbolen
- Epsilon-Transitionen (gestrichelt)
- Self-Loops und Rückkanten
- Interaktive Zoom/Pan-Funktionen

Test-Coverage: ~75% Lines

dslParser.ts

Zweck: Parsing von AEF-Format zu NFA-Objekten

Features:

- 12+ Validierungsregeln
- Metadaten-Extraktion (@NAME, @REGEX)
- Epsilon-Erkennung
- Duplikat-Deduplication
- Detaillierte Fehlerberichte

Test-Coverage: 99.09% Lines, 97.72% Branches

subsetConstruction.ts

Zweck: Implementierung des Subset Construction Algorithmus

Status: In Entwicklung (Sprint 2)

---

4. Datenmodelle

4.1 NFA (Nichtdeterministischer Endlicher Automat)

```
interface NFA {
  name?: string;           // Optionaler Name
  regex?: string;          // Optionaler regulärer Ausdruck
  states: string[];        // Zustandsmenge
```

```
  startState: string;      // Startzustand
  acceptStates: string[];  // Akzeptierende Zustände
  alphabet: string[];      // Eingabealphabet
  transitions: Transition[]; // Übergangsfunktion
  hasEpsilon: boolean;     // ε-Transitionen vorhanden?
}
```

4.2 Transition

```
interface Transition {
  from: string;   // Quellzustand
  to: string;     // Zielzustand
  symbol: string; // Eingabesymbol (oder ε)
}
```

4.3 DFA (Deterministischer Endlicher Automat)

```
interface DFA {
  name?: string;
  regex?: string;
  states: string[];        // Potenzmengen als Zustände
  startState: string;
  acceptStates: string[];
  alphabet: string[];
  transitions: Transition[];
}
```

---

5. AEF-Format Spezifikation

5.1 Syntax

```
# Metadaten (optional)
@NAME <Name des Automaten>
@REGEX <Regulärer Ausdruck>

# Transitionen
<Quellzustand> -<Symbol>> <Zielzustand> [-<Symbol>]...;
```

5.2 Zustandsnotationen

Notation	Bedeutung	Beispiel
`.q0`	Startzustand	`.q0 -a> q1;`
`(q1)`	Akzeptierender Zustand	`q0 -a> (q1);`
`(.q0)`	Start- und Akzeptierender Zustand	`(.q0) -a> q1;`
`q0`	Normaler Zustand	`q0 -a> q1;`

5.3 Beispiel

```
# @NAME Example1_NFA
# @REGEX (1.(0.1)+)|(0.0)

.q0 -1> q1 -0> q2 -1> q3 -0> q4;
```

```
q3 -ε> (q5);
.q0 -0> q6 -0> (q5);
q4 -1> q3;
```

---

6. Qualitätssicherung

6.1 Test-Strategie

Unit Tests

- **dslParser.test.ts:** 25 Tests für Parser-Logik
- **Fokus:** Validierung, Fehlerbehandlung, Edge Cases

Component Tests

- **DSLInput.test.tsx:** 17 Tests für UI-Komponente
- **GraphViewer.test.tsx:** 13 Tests für Visualisierung
- **Fokus:** Benutzerinteraktionen, Rendering, State Management

Integration Tests

- **example-nfa.test.ts:** 17 Tests mit realen Beispielen
- **Fokus:** End-to-End Parsing und Validierung

6.2 Coverage-Ziele

Modul	Lines	Branches	Status
dslParser.ts	99.09%	97.72%	☑ Exzellent
DSLInput.tsx	86.79%	66.66%	☑ Gut
GraphViewer.tsx	~75%	~70%	☑ Akzeptabel

6.3 CI/CD Pipeline

```
stages:
  - install     # Dependencies installieren
  - test        # Unit & Component Tests
  - build       # Production Build
  - deploy      # Deployment (optional)
```

---

7. Deployment

7.1 Entwicklungsumgebung

```
# Installation
npm install

# Development Server
```

```
npm run dev
# → http://localhost:5173

# Tests ausführen
npm test

# Coverage Report
npm run test:coverage
```

7.2 Production Build

```
# Build erstellen
npm run build
# → Ausgabe in dist/

# Build testen
npm run preview
```

7.3 Deployment-Optionen

- **Statisches Hosting:** GitHub Pages, Netlify, Vercel
- **Docker Container:** Nginx + Static Files
- **Uni-Server:** Direktes Deployment auf Webserver

---

8. Projektstatus

8.1 Abgeschlossen (Sprint 1 und Sprint 2)

- ☑ AEF-Format Parser mit strikter Validierung
- ☑ NFA-Visualisierung mit automatischem Layout
- ☑ Datei-Upload und Beispiel-Verwaltung
- ☑ Comprehensive Test-Coverage (55+ Tests)
- ☑ Fehlerberichterstattung mit Zeilennummern

8.2 In Entwicklung (Sprint 3)

- 🔄 Subset Construction Algorithmus
- 🔄 Schrittweise Ausführung mit Animation
- 🔄 DFA-Export und Vergleichsansicht

8.3 Geplant (Sprint 4)

Minimierung von DFAs
Reguläre Ausdrücke → NFA Konvertierung
Interaktive Simulationsumgebung

---

9. Risiken und Herausforderungen

9.1 Technische Risiken

Risiko	Wahrscheinlichkeit	Impact	Mitigation
Performance bei großen NFAs	Mittel	Hoch	Lazy Loading, Virtualisierung
Browser-Kompatibilität	Niedrig	Mittel	Moderne Browser voraussetzen
Layout-Probleme bei komplexen Graphen	Hoch	Mittel	Dagre-Parameter tunen

9.2 Projektrisiken

- **Zeitplan:** Subset Construction komplexer als geplant
- **Testing:** Edge Cases schwer zu identifizieren
- **Usability:** Komplexe UI für Anfänger

---

10. Lessons Learned

10.1 Erfolge

☑ Strikte Validierung verhindert Fehler frühzeitig
☑ Test-First Ansatz erhöht Code-Qualität
☑ ReactFlow + Dagre: Leistungsstarke Graph-Visualisierung

10.2 Verbesserungspotential

⚠️ Frühere User-Tests für besseres UX-Feedback
⚠️ Mehr Beispiel-Dateien für verschiedene Edge Cases
⚠️ Performance-Profiling bei großen Automaten

---

11. Referenzen

- Hopcroft, Motwani, Ullman: Automatentheorie
- ReactFlow Documentation
- Dagre Graph Layout
- Vite Documentation

Dokumentversion: 1.0
Letzte Aktualisierung: Januar 2026